
POSITION: PROTOCOL-AWARE GUARDRAILS ARE NECESSARY FOR MULTI-AGENT SYSTEMS

Anonymous authors

Paper under double-blind review

ABSTRACT

Multi-agent systems (MAS) are quickly becoming the default medium for tool-using AI in products: agents delegate, negotiate, call tools, exchange artifacts, and operate diverse system environments. Yet most deployed guardrails remain single-agent in design, operating on local text or turn-level constraints. This mismatch creates a structural safety gap: in MAS, harm is often *protocol-shaped*, emerging from how messages, roles, capabilities, and state transitions compose across agents over time. We argue for *protocol-aware guardrails*: defenses that treat the MAS as a distributed system with explicit interaction semantics. We propose concrete building blocks, typed interaction protocols, capability/consent tokens bound to provenance, and runtime monitors enforcing global invariants over execution traces, and outline evaluation criteria that reflect deployment constraints such as latency, utility, and composability.

1 INTRODUCTION

Agentic systems are shifting from single, monolithic chats to multi-agent workflows that decompose tasks, discover tools, and exchange intermediate artifacts. This shift is accompanied by rapid progress in agent evaluation (Liu et al., 2024) and multi-agent orchestration patterns (Wu et al., 2023; Park et al., 2023), but also by a growing body of evidence that tool-using agents are vulnerable to prompt injection and related integrity failures (Greshake et al., 2023; Debenedetti et al., 2024; Liu et al., 2023). In parallel, the tool ecosystem itself is expanding: models are trained to invoke thousands of APIs (Qin et al., 2024; Patil et al., 2024) and to decide *when* and *which* tools to use (Huang et al., 2023), increasingly under weak or implicit trust boundaries. A common production response is to place a mature guardrail layer at the user boundary, such as programmable rails for constraining model behavior (Rebedea et al., 2023). However, MAS failures often arise not from any single response, but from how agent outputs and tool results propagate through delegation chains and across components. We argue that MAS safety therefore requires guardrails that reason over *multi-step interaction structure* rather than isolated turns.

2 THE LIMITS OF SINGLE-AGENT GUARDRAILS IN MULTI-AGENT SETTINGS

Most deployed guardrails for language models operate at a local level: filtering disallowed content, prompt sanitization, or constraining output validation (Rebedea et al., 2023). This orientation is appropriate when the unit of risk can be confined to a single model output. In contrast, multi-agent systems involve coordinated execution across multiple agents that exchange messages and artifacts, invoke tools, and evolve shared state over time, often under implicit authority assumptions that cause intermediate outputs to be trusted by downstream agents in the network. As a result, safety violations may not be attributable to any single local interaction: behavior that appears acceptable in isolation can lead to policy violations at the system level. This mirrors classic results from distributed systems, where correctness and safety are defined over global execution traces rather than individual events (Lamport, 1978; Tanenbaum & Van Steen, 2007). In multi-agent systems, guardrails applied only at the user interface or text layer fundamentally cannot capture failures that arise from routing, delegation, and state evolution within the system.

Table 1: Why MAS needs protocol-aware guardrails: single-agent controls do not align with MAS failure units.

Control layer	Safety unit	Common MAS failure it misses
Content filtering / refusal	Turn / message	Multi-step harms that emerge only after delegation or tool execution
Programmable rails (Rebedea et al., 2023)	Dialogue policy	Cross-agent state confusion; “trust relabeling” of artifacts
Tool allow/deny	Tool call	Cascading injection via “trusted” intermediate summaries (Greshake et al., 2023; Debenedetti et al., 2024)
Task benchmarks (Liu et al., 2024)	End-task success	Authority and integrity violations across agent graphs
Protocol-aware guardrails (this paper)	Execution trace	Directly targets delegation, authority, and state transitions

3 WHEN TRUST SPREADS: FAILURE MODES THAT ONLY SHOW UP ACROSS THE TRACE

Many distinct failure modes exist in multi agent systems, and the key point is that harm often shows up only when we look at the full execution trace across agents, tools, and state updates, not any single message. A first failure mode is cascading injection, where untrusted data becomes dangerous only after an upstream agent transforms or summarizes it and a downstream agent treats it as trusted reasoning. In multi agent settings, one injected payload can move through normal delegation and artifact reuse and then grow in impact as more agents accept the polluted output (Sharma et al., 2025). This matches evidence that indirect prompt injection can compromise real LLM integrated applications because models struggle to separate instructions from data (Greshake et al., 2023; Liu et al., 2023), and agent systems make this worse by adding many ingestion points, tool surfaces, and trust boundaries (Debenedetti et al., 2024).

Huang and collaborators in the OWASP agentic security effort list concrete threat classes that become more severe in multi agent graphs, such as goal manipulation, unsafe delegation, tool misuse, and cross component confusion (OWASP Foundation, 2025). Such cases follow from common design patterns like putting the user goal into a persistent system prompt, letting one agent write instructions for another agent, and letting agents act on observations from tools and web content. In practice, this means a small early mistake or a small early attack can change many later steps, even if each step looks reasonable on its own.

A second failure mode is accountability collapse under delegation. When planning and execution are split across agents, the context that caused a bad action can be far away from the action itself, so it is hard to answer who decided and who executed, which makes debugging and fixing the system much harder (Debenedetti et al., 2024). A third failure mode is protocol logic weakness. Many systems encode authority, intent, and role meaning only in natural language, so an agent can confuse state, treat data as instructions, or gain extra power through delegation. This is a control plane problem, similar to distributed systems where safety depends on properties of the whole history, not a single event (Lamport, 1978; Tanenbaum & Van Steen, 2007). Finally, the MASEC work makes this trace view explicit by pushing threat modeling that follows how trust and authority move through agent to agent and agent to tool pathways, which directly targets cascading injection, lost provenance, and unsafe delegation patterns (Krawiecka & Schroeder de Witt, 2025).

4 REQUIREMENTS FOR MULTI-AGENT GUARDRAILS

A guardrail layer for MAS must support policies defined over interaction semantics rather than only over text. Concretely, defenses should be able to refer to agent roles, message types, delegation edges, tool invocations, and state transitions, enabling constraints over execution traces rather than isolated turns. Moreover, authority should not be implicit: tool privileges must be bound to explicit, auditable grants, and the system should record provenance that ties actions to upstream intent and policy checks. Capability-based security offers a useful lens here: the system should treat au-

Table 2: Example invariant library for protocol-aware guardrails (illustrative).

Invariant	Enforced on	Intuition / what it prevents
No privilege escalation	Delegation edges	Downstream agents cannot gain new tool rights without an explicit grant (Dennis & Van Horn, 1966).
Untrusted-to-trusted barrier	Artifact relabels	Summaries of external content cannot be re-classified as “policy-approved” inputs without checks (Greshake et al., 2023; Chen et al., 2025).
Two-person rule for high risk	Tool calls	Sensitive actions require independent approval by a separate agent role.
Tool-call justification	Tool calls	Each action must cite an upstream intent and policy check for auditability (Debenedetti et al., 2024).
State-machine conformance	Message flow	Prevents role confusion (e.g., tool results treated as instructions) via typed protocols.

thority as a first-class object that can be bounded, delegated, and audited, rather than inferred from conversational context (Dennis & Van Horn, 1966). Finally, enforcement must be online and compositional: violations of global invariants should be prevented at runtime with bounded overhead, rather than merely detected after-the-fact. This connects MAS guardrails to runtime verification, which develops practical monitoring techniques for safety properties over traces (Havelund & Rosu, 2004; Bauer et al., 2006).

5 PROTOCOL-AWARE GUARDRAILS AS A POSITION

We propose *protocol-aware guardrails*, a safety layer that treats a multi-agent application as a distributed system whose behavior is governed by explicit interaction protocols.

Typed interaction semantics. Messages and artifacts are labeled with declared types (e.g., plan, tool request, tool result, approval request, external content), and allowed transitions are defined by a protocol. This makes interaction structure machine-checkable rather than implicit in free-form text. Concretely, protocol typing converts “prompt-level contracts” into enforceable interaction contracts, aligning with systems architectures that separate reasoning from external knowledge and actions (Karpas et al., 2022; Yao et al., 2022b).

Capability and consent tokens bound to provenance. Effectful tool actions require explicit grants that encode scope, duration, and upstream intent (who requested, what justification, what policy checks passed), preventing silent privilege escalation and making delegation auditable. This responds to an empirical reality: modern agents are trained to call tools at scale (Qin et al., 2024; Patil et al., 2024) and to combine reasoning with acting (Yao et al., 2022b), which increases the attack surface and amplifies the cost of implicit authorization.

Runtime monitoring of invariants over execution traces. The system enforces global properties such as non-escalation of privileges, separation of duties for high-risk actions, and constraints on relabeling untrusted artifacts as trusted reasoning. These invariants can be expressed in lightweight temporal formalisms and enforced through monitors that interpose on inter-agent messaging and tool invocation (Havelund & Rosu, 2004; Bauer et al., 2006). The key claim is that MAS safety should be evaluated and enforced at the level of *trace semantics*: the set of permissible histories under a protocol, not only the set of permissible utterances.

5.1 A MINIMAL ARCHITECTURE SKETCH

A practical implementation can be lightweight: (i) a schema for typed messages/artifacts, (ii) a policy engine mapping protocol states to permitted transitions and required checks, (iii) a capability manager minting short-lived tokens that bind tool authority to provenance, and (iv) a runtime monitor that observes the event stream (messages, delegation, tool I/O) and blocks transitions that violate invariants. Importantly, the monitor can sit *outside* the LLM, enabling defense-in-depth even when

the agent model is compromised by injected instructions. Recent work suggests that defenses based on imposing structure at the boundary (e.g., structured queries) can materially change the injection surface, supporting the broader thesis that “shape” matters for safety (Chen et al., 2025).

5.2 WHY THIS IS NOT JUST “MORE ALIGNMENT”

Training-time alignment (e.g., RLHF and constitutional approaches) improves average helpfulness and refusal behavior (Ouyang et al., 2022; Bai et al., 2022), and remains valuable as a baseline. But protocol-aware guardrails target a different failure class: *distributed integrity* under composition. Even a well-aligned agent can be made to misuse authority if the system allows untrusted data to flow into privileged decision points or allows delegation to silently expand capabilities (DeBenedetti et al., 2024; Greshake et al., 2023). The proposed approach therefore complements alignment by hardening the control plane that governs multi-agent execution.

6 EVALUATION CRITERIA FOR MAS GUARDRAILS

Failures in multi-agent systems depend on execution traces and agent topology rather than isolated interactions, requiring evaluation criteria that go beyond single-task success. In particular, MAS guardrails must be assessed on (i) their ability to prevent violations that arise across delegation chains and artifact flows, (ii) portability across agent architectures and coordination patterns, and (iii) the security–utility trade-offs they introduce. Existing agentic benchmarks provide useful starting points: task-oriented suites measure performance under interactive conditions (Liu et al., 2024), long-horizon web environments expose how local errors accumulate over time (Zhou et al., 2023; Deng et al., 2023; Yao et al., 2022a), and security-focused settings stress defenses under injection-style threats (DeBenedetti et al., 2024; Greshake et al., 2023; Liu et al., 2023). However, these evaluations largely treat safety as a proxy for task failure, rather than as a property of system execution.

We argue that MAS guardrail evaluation should instead elevate *invariant violations* to first-class outcomes. Concretely, evaluations should (a) measure violations of protocol- and authority-level invariants independently of task success, (b) report latency and cost overhead introduced by enforcement, (c) quantify utility impact on benign workloads, and (d) localize violations to specific protocol steps to support debugging and remediation. One practical metric is a *trace safety rate*, defined as the fraction of execution traces whose event logs satisfy a specified invariant library (Table 2), reported alongside conventional success metrics. This distinction is critical in real-world deployments: systems may be acceptable to ship if they fail tasks gracefully, but not if they violate invariants such as unauthorized tool use or improper delegation, even when the final output appears correct.

7 CONCLUSION

Single-agent guardrails remain valuable, but they are insufficient for multi-agent systems where harms emerge from protocol semantics, delegation structure, and stateful composition. We argue for protocol-aware guardrails built on typed interactions, explicit capability/consent tokens, and runtime monitors enforcing global invariants over execution traces. This framing offers a concrete research and engineering path for “agents in the wild”: safer composition across heterogeneous agent graphs without collapsing into brittle keyword filters or purely post-hoc auditing.

REFERENCES

- Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.
- Andreas Bauer, Martin Leucker, and Christian Schallhart. Runtime verification for LTL and TLTL. In *Proceedings of the International Conference on Runtime Verification*, 2006.
- Sizhe Chen et al. Struq: Defending against prompt injection with structured queries. In *Proceedings of the USENIX Security Symposium*, 2025.

216 Edoardo DeBenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer Kellner, Marc Fischer, and Florian
217 Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses
218 for LLM agents. *arXiv preprint arXiv:2406.13352*, 2024.

219

220 Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and
221 Yu Su. Mind2web: Towards a generalist agent for the web. *arXiv preprint arXiv:2306.06070*,
222 2023.

223

224 Jack B. Dennis and Earl C. Van Horn. Programming semantics for multiprogrammed computations.
225 *Communications of the ACM*, 9(3):143–155, 1966.

226

227 Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario
228 Fritz. Not what you’ve signed up for: Compromising real world LLM integrated applications
229 with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023.

230

231 Klaus Havelund and Grigore Rosu. Efficient monitoring of safety properties. *International Journal
232 on Software Tools for Technology Transfer*, 6(2):158–173, 2004.

233

234 Yue Huang, Jiawen Shi, Yuan Li, Chenrui Fan, Siyuan Wu, Qihui Zhang, Yixin Liu, Pan Zhou, Yao
235 Wan, Neil Zhenqiang Gong, and Lichao Sun. Metatool benchmark for large language models:
236 Deciding whether to use tools and which to use. *arXiv preprint arXiv:2310.03128*, 2023.

237

238 Ehud Karpas, Omri Abend, Yonatan Belinkov, Barak Lenz, Opher Lieber, Nir Ratner, Yoav Shoham,
239 Hofit Bata, Yoav Levine, Kevin Leyton Brown, et al. MRKL systems: A modular, neuro sym-
240 bolic architecture that combines large language models, external knowledge sources and discrete
241 reasoning. *arXiv preprint arXiv:2205.00445*, 2022.

242

243 Klaudia Krawiecka and Christian Schroeder de Witt. Extending the owasp multi agentic sys-
244 tem threat modeling guide: Insights from multi agent security research. *arXiv preprint
245 arXiv:2508.09815*, 2025.

246

247 Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications
248 of the ACM*, 21(7):558–565, 1978.

249

250 Xiao Liu et al. Agentbench: Evaluating LLMs as agents. In *Proceedings of ICLR*, 2024. *arXiv
251 preprint arXiv:2308.03688*.

252

253 Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Zihao Wang, Xiaofeng Wang, Tianwei Zhang,
254 Yepang Liu, Haoyu Wang, Yan Zheng, and Yang Liu. Prompt injection attack against LLM
255 integrated applications. *arXiv preprint arXiv:2306.05499*, 2023.

256

257 Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong
258 Zhang, et al. Training language models to follow instructions with human feedback. In *Advances
259 in Neural Information Processing Systems*, 2022.

260

261 OWASP Foundation. Multi agentic system threat model-
262 ing guide v1.0. [https://genai.owasp.org/resource/
263 multi-agentic-system-threat-modeling-guide-v1-0/](https://genai.owasp.org/resource/multi-agentic-system-threat-modeling-guide-v1-0/), 2025. Accessed
264 2026 02 05.

265

266 Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and
267 Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint
268 arXiv:2304.03442*, 2023.

269

270 Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model
271 connected with massive apis. In *Advances in Neural Information Processing Systems*, 2024.
272 *arXiv:2305.15334*.

273

274 Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, et al.
275 ToolLLM: Facilitating large language models to master 16000+ real world apis. In *Proceedings
276 of ICLR*, 2024.

270 Traian Rebedea, Razvan Dinu, Makesh Narsimhan Sreedhar, Christopher Parisien, and Jonathan Co-
271 hen. Nemo guardrails: A toolkit for controllable and safe LLM applications with programmable
272 rails. *Proceedings of EMNLP: System Demonstrations*, pp. 431–445, 2023.

273 Gauri Sharma, Vidhi Kulkarni, Miles King, and Ken Huang. Towards unifying quantitative security
274 benchmarking for multi agent systems. *arXiv preprint arXiv:2507.21146*, 2025.

275 Andrew S. Tanenbaum and Maarten Van Steen. *Distributed Systems: Principles and Paradigms*.
276 Prentice Hall, 2007.

277 Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun
278 Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and
279 Chi Wang. Autogen: Enabling next gen LLM applications via multi agent conversation. *arXiv*
280 *preprint arXiv:2308.08155*, 2023.

281 Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable
282 real world web interaction with grounded language agents. In *Advances in Neural Information*
283 *Processing Systems*, 2022a. *arXiv:2207.01206*.

284 Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao.
285 React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*,
286 2022b.

287 Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng,
288 Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic
289 web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.

290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323